

DESAFÍO I — SWEET CRUSH

**Implementación de un tablero dinámico mediante manipulación de bits en
C++**

**Universidad de Antioquia
Informática II – Semestre 2026-2**

**Estudiante: Shirly Maria Atehortua Velásquez
Sección: 4
Fecha: 20 de septiembre de 2026**

1. Introducción

El presente informe describe el proceso de análisis, diseño e implementación del Desafío I de Informática II, cuyo objetivo es desarrollar un juego basado en la lógica de Sweet Crush utilizando C++.

El desarrollo se realizó bajo restricciones relacionadas con la memoria dinámica, los punteros, los arreglos y las operaciones a nivel de bits. Una de las principales dificultades fue representar cada ficha con solo tres bits y almacenar todo el tablero como una secuencia compacta de bits.

La solución permite crear un tablero de dimensiones variables, generar fichas aleatorias, seleccionarlasy eliminarlas, detectar combinaciones horizontales y verticales, eliminar las fichas correspondientes, aplicar la gravedad, generar nuevas fichas y procesar cascadas. Además, permite modificar dinámicamente el tablero mediante la adición y eliminación de filas y columnas.

2. Análisis del problema

a. Características principales: El problema plantea la construcción de un tablero dinámico compuesto por seis tipos distintos de fichas. Cada ficha debe representarse mediante exactamente tres bits. Se utilizan seis valores para las fichas y se reserva 000 para representar una posición vacía durante los procesos de eliminación y reorganización.

Ficha	Código binario
1	001
2	010
3	011
4	100
5	101
6	110
Vacío	000

b. Representación del tablero: Se decidió utilizar un único puntero de tipo unsigned char* para representar el tablero. Las fichas se almacenan de forma consecutiva en grupos de tres bits.

unsigned char* tablero;

La cantidad mínima de memoria se calcula a partir de las dimensiones:

cantidad de bits = ancho × alto × 3

cantidad de bytes = (cantidad de bits + 7) / 8

La representación permite que una ficha comience en un byte y termine en el siguiente, por lo que fue necesario desarrollar algoritmos para extraer y modificar fichas mediante desplazamientos y máscaras.

3. Consideraciones para la solución propuesta

Durante el análisis se consideraron diversas posibilidades para representar el tablero. Utilizar un byte completo por ficha no cumpliría con el requisito de usar tres bits por posición. Por esta razón se decidió trabajar directamente sobre una secuencia compacta de bits.

Code/

- main.cpp
- Tablero.h
- Tablero.cpp
- Combinaciones.h
- Combinaciones.cpp

main.cpp se utiliza para la interacción con el usuario y el control general. Tablero.h y Tablero.cpp contienen las operaciones de memoria, almacenamiento, acceso, modificación y reorganización. Combinaciones.h y Combinaciones.cpp contienen la lógica de detección, eliminación y de cascadas.

4. Diseño de la solución

a. Creación del tablero

El tablero se crea dinámicamente. La cantidad de memoria se determina según las dimensiones actuales y los 3 bits por ficha.

```
unsigned char* crearTablero(int ancho, int alto);
```

b. Generación de fichas

Las fichas se generan mediante valores entre 1 y 6:

```
return rand() % 6 + 1;
```

De esta forma, cada una de las seis fichas puede seleccionarse tanto durante la generación inicial como durante el relleno.

5. Acceso a una ficha

Para encontrar una ficha se convierte la posición (fila, columna) en un índice lineal, se multiplica por tres para obtener la posición inicial de sus bits, se determina el byte donde comienza y el desplazamiento dentro del byte. Si los tres bits están dentro del mismo byte se extraen directamente; si cruzan el límite de un byte, se toma una parte de cada byte y se combinan.

```
valor = (tablero[byte] >> desplazamiento) & 7;
```

6. Modificación de una ficha

La función `modificarFicha` utiliza máscaras para limpiar los bits correspondientes a la ficha anterior y, posteriormente, establecer el nuevo valor. Si la ficha cruza el límite de un byte, sus bits se dividen entre ambos bytes.

7. Detección de combinaciones

La detección se implementó mediante dos recorridos: uno horizontal, que recorre filas en busca de tres fichas iguales consecutivas, y otro vertical, que recorre columnas. Las posiciones encontradas se almacenan en un arreglo auxiliar llamado `eliminadas` antes de modificar el tablero.

8. Combinaciones simultáneas

El marcador permite identificar fichas que pertenecen simultáneamente a una combinación horizontal y a una vertical. Si una posición se encuentra nuevamente, permanece marcada y no se elimina dos veces.

```
eliminadas[posicion] = 1;
```

9. Eliminación y gravedad

Las posiciones marcadas se convierten en 0, que representa una posición vacía. Después, se aplica la gravedad columna por columna desde la parte inferior, desplazando las fichas hacia las posiciones disponibles. Finalmente se generan nuevas fichas para las posiciones vacías.

10. Procesamiento de cascadas

El procesamiento de cascadas utiliza un ciclo que continúa mientras existan combinaciones. El procedimiento es: detectar combinaciones, eliminarlas, aplicar la gravedad, rellenar el tablero y volver a buscar combinaciones. Cuando ya no existen nuevas combinaciones, el tablero se considera estable.

Detectar combinaciones

-

Eliminar fichas

-

Aplicar gravedad

-
 Rellenar tablero
 -
 Buscar nuevas combinaciones
 -
 ¿Existen?
 - -
 Sí No
 - -
 Repetir Finalizar

Durante las pruebas, se observó una eliminación que provocó seis cascadas consecutivas.

11. Modificación dinámica del tablero

El programa permite agregar y eliminar filas y columnas, incluso en posiciones intermedias. Para agregar, se crea un nuevo bloque de memoria, se conservan las fichas existentes y se generan nuevas fichas en la posición indicada. Para eliminar, se copian las fichas que deben conservarse y se reorganizan según las nuevas dimensiones.

12. Gestión de memoria

Al eliminar una fila o una columna, se calcula la cantidad de memoria necesaria para las nuevas dimensiones y se compara con la capacidad disponible. Cuando el porcentaje de utilización cae por debajo del 65 %, se libera el bloque anterior y se utiliza uno de menor tamaño.

13. Visualización del tablero

El programa permite visualizar el tablero en formato de fichas y en formato binario.

TABLERO EN DECIMAL:

5 2 5 2 5 4
 4 6 4 6 4 2

TABLERO EN BINARIO:

101 010 101 010 101 100
 100 110 100 110 100 010

La visualización binaria permite comprobar que cada ficha corresponde a una representación de 3 bits.

14. Estadísticas del juego

El programa mantiene información sobre el ancho y el alto actuales, las eliminaciones realizadas por el usuario, las fichas eliminadas, las combinaciones detectadas, las cascadas de la última eliminación y la puntuación.

15. Sistema de puntuación

Se definió un sistema de puntuación propio:

10 puntos por cada eliminación realizada por el usuario.

10 puntos por cada ficha eliminada mediante combinación.

20 puntos por cada combinación detectada.

30 puntos por cada cascada procesada.

Revisar esta sección antes de entregarla para que coincida exactamente con las reglas finales documentadas.

16. Problemas encontrados durante el desarrollo

El desarrollo de este proyecto representó un reto importante para mí, principalmente porque al comienzo confundí el desafío con el proyecto de Tetris en el que había trabajado anteriormente. Esto hizo que, inicialmente, partiera de bases que no correspondían plenamente al objetivo de Sweet Crush, por lo que tuve que reorganizar el proyecto y comenzar nuevamente desde las bases que ya había construido. Aunque al principio esto fue frustrante, también me permitió entender mejor qué partes del código debía conservar, cuáles modificar y cómo adaptar la solución al problema planteado.

Uno de los temas que más dificultades me generó fue el manejo de punteros y de la memoria dinámica. Al principio tenía una idea diferente de cómo representar el tablero, pero las condiciones del proyecto exigían trabajar con un único puntero de tipo unsigned char*, sin utilizar punteros dobles. Esto me obligó a comprender realmente cómo podía almacenar todo el tablero en una secuencia continua de memoria y cómo acceder a cada ficha mediante posiciones de bits. No fue solo cambiar el tipo de una variable: tuve que entender cómo calcular la posición de cada ficha, cómo encontrar el byte correspondiente y cómo extraer o modificar solo los tres bits que correspondían a cada ficha.

También fue un reto desarrollar la lógica de las combinaciones y de las cascadas. No era suficiente con detectar tres fichas iguales y eliminarlas. Después de una eliminación, era necesario aplicar la gravedad, rellenar los espacios vacíos y comprobar si las nuevas fichas generaban nuevas combinaciones. Esto me llevó a desarrollar una lógica que permitiera

repetir el proceso hasta que el tablero quedara estable. En este punto entendí mejor la importancia de dividir el problema en funciones y asignar a cada parte del programa una responsabilidad específica.

Durante el desarrollo, también encontré errores que parecían pequeños, pero que podían detener por completo la ejecución o la compilación del programa. Uno de los casos que más recuerdo fue un problema relacionado con el tabulador y el formato del código, en el que el error que aparecía parecía afectar a una parte más amplia del programa, pero finalmente se debía a una sola línea. Esto me enseñó que cuando un programa tiene muchos módulos y funciones, el lugar donde aparece el error no siempre coincide con el lugar donde está el problema, por lo que fue necesario revisar con cuidado el código y no asumir de inmediato cuál era la causa.

Otro reto fue el mensaje de advertencia que todavía no he logrado resolver por completo. Aunque el programa funciona y las pruebas realizadas han dado los resultados esperados, este mensaje me permitió identificar que todavía hay aspectos del manejo de la memoria que debo seguir revisando. En lugar de simplemente ignorarlo, lo considero parte del proceso de aprendizaje porque me hizo revisar nuevamente cómo el compilador y las herramientas de análisis verifican los accesos a la memoria.

Además, una dificultad importante fue desarrollar el proyecto sin apoyarme en librerías, vectores, clases, estructuras u otras herramientas que simplificaran el manejo de los datos, debido a las restricciones establecidas para el desafío. Esto fue especialmente importante porque muchas soluciones que se encuentran fácilmente o que una IA puede sugerir utilizan tipos vector, string, clases u otras estructuras que en este proyecto no están permitidos. Por esta razón, no podía limitarme a copiar una solución que funcionara: tenía que comprender qué hacía el código y buscar una forma de implementarlo utilizando los conceptos estudiados en la asignatura.

En este proceso también comprendí que una solución generada automáticamente no necesariamente resulta útil si no se entiende. La inteligencia artificial puede proponer código rápidamente, pero en varias ocasiones tuve que detenerme, revisar la lógica, preguntar qué hacía cada parte y adaptarla a las restricciones del proyecto. Esto fue especialmente importante con los punteros, la representación de 3 bits por ficha, la memoria dinámica y el procesamiento en cascada. El hecho de haber trabajado estos conceptos durante el curso me permitió relacionarlos con el problema y entender por qué determinadas soluciones funcionaban y otras no.

Finalmente, considero que una de las mayores dificultades del proyecto fue que no se trataba únicamente de lograr que el programa compilara. Tuve que enfrentarme a errores, reorganizar el código, modificar la representación del tablero, entender la memoria y comprobar que cada nueva funcionalidad no dañara las anteriores. Hubo momentos en los que me costó bastante encontrar la forma correcta de implementar lo que se pedía y tuve que exigirme mucho para seguir adelante. Sin embargo, precisamente esas dificultades hicieron que el proyecto fuera una experiencia de aprendizaje importante, porque me permitió llevar a la práctica conceptos que habíamos estudiado en clase y comprenderlos mediante su implementación, especialmente el uso de punteros, memoria dinámica, arreglos, operaciones de bits y modularización.

17. Evolución de la solución

La solución del proyecto se implementó de forma gradual, mediante diversas pruebas y rectificaciones, hasta incluir las funciones principales del reto. El historial de commits muestra la evolución del proyecto desde su primera estructura hasta una versión funcional de Sweet Crush.

En primer lugar, se llevaron a cabo la creación y la organización de los archivos del tablero y del repositorio. Se inició la implementación del tablero dinámico tras reestructurar el proyecto para centrarse específicamente en Sweet Crush.

La solución del proyecto se implementó poco a poco, mediante varias pruebas y correcciones, hasta incorporar las funciones principales del desafío. El registro de commits refleja el progreso del proyecto desde su estructura inicial hasta una versión operativa de Sweet Crush.

Primero, se crearon y organizaron los archivos del repositorio y del tablero. Después de reestructurar el proyecto para enfocarse en Sweet Crush, se comenzó a implementar el tablero dinámico.

Esta fase está relacionada con el commit "Estadísticas y puntuación, pendiente lo que es tablero en binario". Después, el repositorio entró en una fase conocida como "Desafío en etapa final".

18. Pruebas realizadas

Prueba de dimensiones

Se verificó la creación de tableros de distintos tamaños.

Prueba de almacenamiento

Se verificó que las fichas podían almacenarse y recuperarse correctamente mediante tres bits.

Prueba de visualización

Se comprobó que el formato decimal coincidía con la representación binaria de 3 bits.

Prueba de eliminación

Se verificó la eliminación de una ficha seleccionada por el usuario.

Prueba de combinaciones

Se verificaron combinaciones horizontales y verticales.

Prueba de cascadas

Se realizó una prueba en la que una eliminación provocó seis cascadas consecutivas.

Prueba de modificaciones dinámicas

Se probaron modificaciones en filas y columnas en posiciones intermedias.

Prueba de memoria

Se verificó el comportamiento del programa al reducir el porcentaje de utilización de la memoria y el límite al 65 %.

19. Resultados obtenidos

Una de las pruebas integrales produjo el siguiente resultado:

Ancho actual: 23

Alto actual: 14

Eliminaciones realizadas por el usuario: 1

Fichas eliminadas: 96

Combinaciones detectadas: 35

Cascadas de la última eliminación: 6

Puntuacion: 1840

El resultado permitió comprobar la integración de la eliminación, las combinaciones, las cascadas y el sistema de puntuación. Además, la visualización binaria mostró correctamente la correspondencia entre las fichas y sus códigos de tres bits.